SciForma-Base

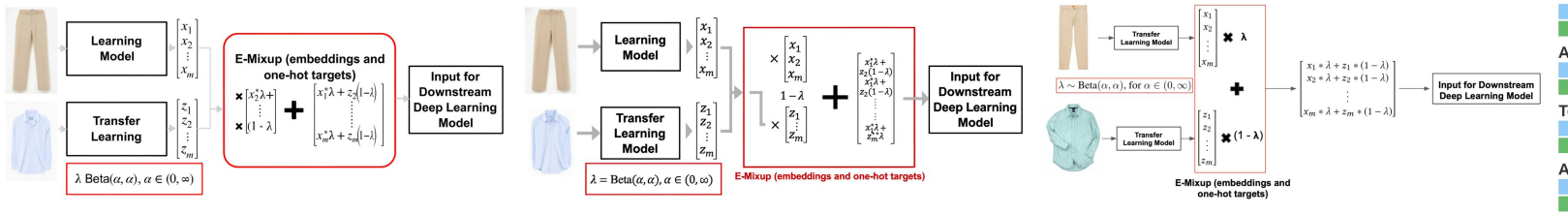
SciForma-9B (+M-DPO)

Ground Truth

Structural Inventory

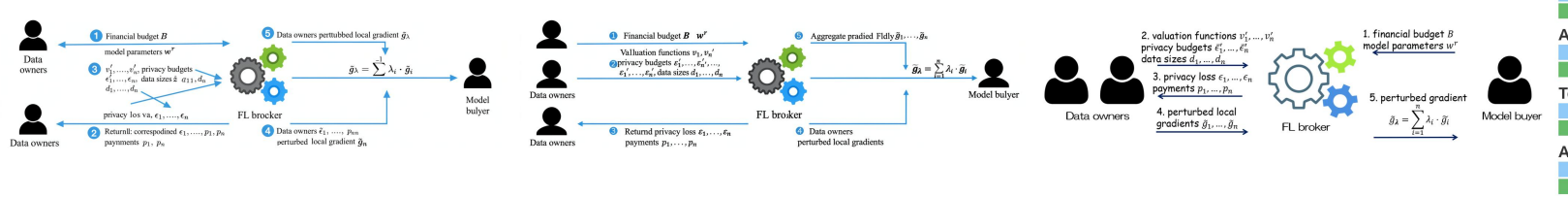
SFT
M-DPO

Prompt: The figure illustrates the E-Mixup augmentation technique applied to embedding representations derived from two distinct input images: a pair of beige trousers and a light blue striped shirt. The global layout is horizontal and sequential, depicting a data processing pipeline from left to right. On the far left, the two images serve as initial inputs, each feeding into a separate 'Transfer Learning Model' block represented as a rectangular box with black borders and bold, centered text. These models extract feature embeddings, denoted as column vectors [x□, x□, ..., x□] for the trousers and [z□, z□, ..., z□] for the shirt. A central red-bordered box labeled 'E-Mixup (embeddings and one-hot targets)' contains the core augmentation operation. Inside this box, the two embedding vectors are scaled by weights λ and (1−λ), respectively, where λ is sampled from a Beta(α, α) distribution with α □ (0, ∞), as indicated by a red-bordered text box below the trousers. The scaling is visually represented by multiplication symbols (×) next to each vector. A large black plus sign (+) between the scaled vectors indicates element-wise addition. The result is a new mixed embedding vector shown as [x□*λ + z□*(1−λ), x□*λ + z□*(1−λ), ..., x□*λ + z□*(1−λ)], displayed as a column vector. An arrow leads from this mixed embedding to a final rectangular box labeled 'Input for Downstream Deep Learning Model', indicating that the augmented representation is fed into a subsequent model. The entire process is designed to augment both embedding inputs and their corresponding one-hot target vectors using the same mixing strategy, as noted in the figure caption. All text elements are in black sans-serif font, and arrows are solid gray lines with arrowheads pointing right, guiding the flow of data through the pipeline.



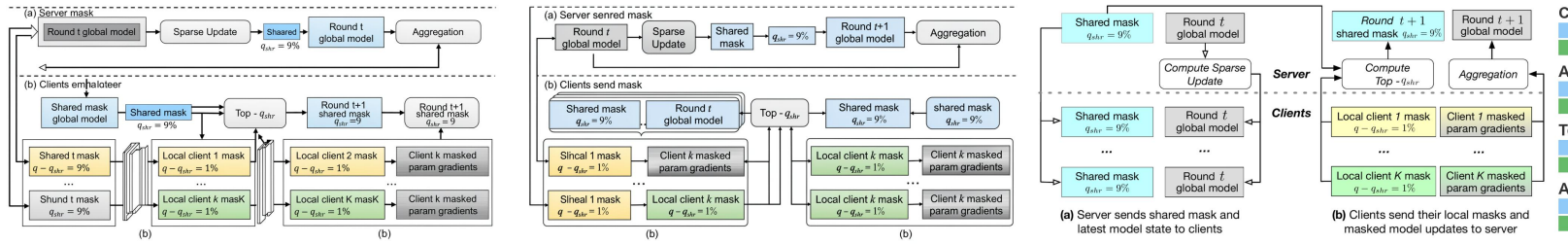
Component (C)	0.714	0.857 +20.0%
Arrow (A)	1.000	1.000 +0.0%
Text (T)	0.292	0.792 +171.2%
Avg	0.669	0.883 +32.1%

Prompt: The figure illustrates the FL-Market Trading Framework, a federated learning-based market mechanism involving data owners, an FL broker, and a model buyer. The global layout is horizontal, depicting a three-party interaction flow from left to right: Data owners on the far left, the FL broker in the center, and the Model buyer on the far right. Each party is represented by a black silhouette icon labeled accordingly. The central component is the FL broker, symbolized by three interlocking gears — one large gray gear and two smaller gears in green and blue — indicating computational processing and coordination. This module acts as the intermediary between data owners and the model buyer. The workflow proceeds in five numbered steps, indicated by directional arrows and labeled text: 1. From the Model buyer to the FL broker: The model buyer sends financial budget B and model parameters w^r. This step initiates the process by specifying resource constraints and initial model state. 2. From Data owners to the FL broker: Data owners provide valuation functions v_1', ..., v_n', privacy budgets ε_1', ..., ε_n', and data sizes d_1, ..., d_n. These inputs represent each owner's utility function, privacy constraints, and data contribution size. 3. From the FL broker back to Data owners: The broker returns privacy loss values ε_1, ..., ε_n and corresponding payments p_1, ..., p_n. This step reflects the outcome of the market mechanism, where privacy costs are quantified and compensated. 4. From Data owners to the FL broker: Data owners send perturbed local gradients g̃_1, ..., g̃_n. These gradients are computed locally on their data and perturbed for privacy preservation before transmission. 5. From the FL broker to the Model buyer: The broker aggregates these gradients into a single perturbed gradient g̃_λ using the formula g̃_λ = Σ_{i=1}^{n} λ_i · g̃_i, where λ_i represents weighting coefficients. This aggregated gradient is then sent to the model buyer for model update. All arrows are solid blue lines with arrowheads indicating direction. Text labels are placed adjacent to or above/below the arrows, clearly marking each step. The figure emphasizes a bidirectional exchange between data owners and the broker, while the model buyer only communicates unidirectionally with the broker. The visual design uses minimalistic icons and clear typography to convey the structured, privacy-aware trading process within a federated learning context.



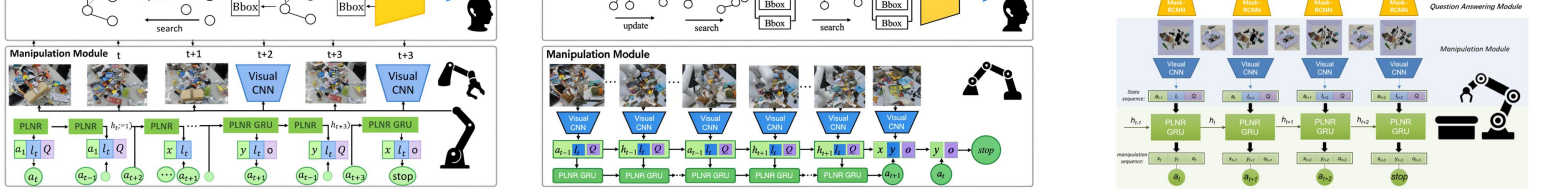
Component (C)	0.833	0.833 +0.0%
Arrow (A)	0.300	1.000 +233.3%
Text (T)	1.000	1.000 +0.0%
Avg	0.711	0.944 +32.8%

Prompt: The figure illustrates a two-phase federated learning framework with a mask-shifting mechanism, designed to enable efficient communication and sparse model updates across clients and a central server. The diagram is divided into two main parts: (a) Server sends shared mask and latest model state to clients, and (b) Clients send their local masks and masked model updates to server. A dashed horizontal line separates the server components at the top from the client components below, indicating the distributed nature of the system. In part (a), the server holds a 'Round t global model' (gray rectangle) and computes a 'Sparse Update' (rounded rectangle) using this model. From this computation, a 'Shared mask' (light blue rectangle) is generated, labeled with q_shr = 9%. This shared mask is then broadcasted to all clients. Each client receives both the shared mask and a copy of the Round t global model. The clients are represented as multiple identical blocks, each containing a 'Shared mask q_shr = 9%' and 'Round t global model', indicating uniform distribution of these elements. In part (b), the process continues with the clients performing local computations. Each client generates a 'Local client k mask' (yellow for client 1, green for client K), labeled with q - q_shr = 1%, meaning the local mask covers the remaining 1% of parameters not covered by the shared mask. Using this combined mask (shared + local), each client computes 'Client k masked param gradients' (gray rectangles with gradient shading). These local masks and masked gradients are then sent back to the server. On the server side in part (b), the incoming local masks and gradients are processed. The server computes 'Top - q_shr' (rounded rectangle), which likely refers to selecting the top q_shr percentage of parameters for the next shared mask, based on the aggregated information from all clients. This results in a new 'Round t+1 shared mask q_shr = 9%', which is then used to compute the 'Round t+1 global model' via an 'Aggregation' step (rounded rectangle). The aggregation step combines the masked gradients from all clients to update the global model. The figure uses color coding to differentiate components: light blue for shared masks, gray for global models and gradients, yellow and green for local client masks, and rounded rectangles for computational steps. Arrows indicate data flow: from server to clients in (a), and from clients to server in (b). A feedback loop connects the new shared mask and global model back to the start of the next round, completing the iterative cycle. The caption specifies that the total sparsity is q = 10%, composed of q_shr = 9% (shared) and 1% (local per client), and notes a potential ambiguity regarding how the new shared mask is derived — it should logically be computed from both the previous shared mask and the incoming local masks.



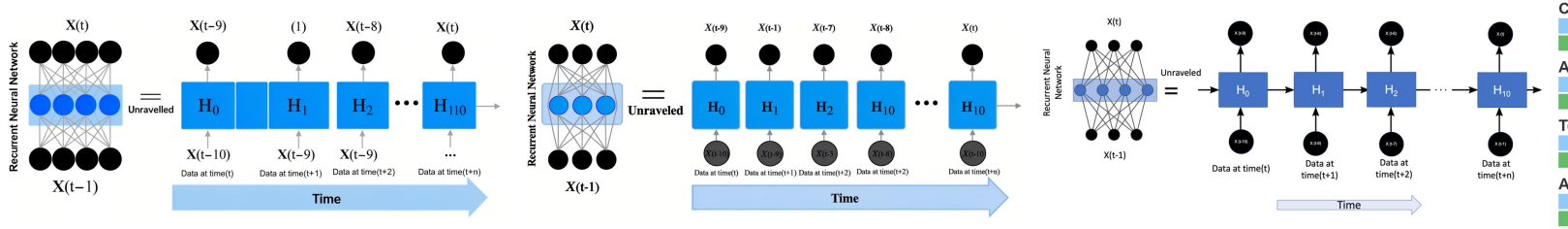
Component (C)	0.650	0.675 +3.8%
Arrow (A)	0.250	0.469 +87.6%
Text (T)	0.500	1.000 +100.0%
Avg	0.467	0.715 +53.1%

Prompt: The figure illustrates the architecture of a Multi-Query Answering (MQA) model, divided into two main components: a Question Answering Module at the top and a Manipulation Module at the bottom. The overall layout is horizontal and sequential, progressing from left to right across four time steps (t, t+1, t+2, t+3), with each step representing an iteration of the system's operation. In the upper section, the Question Answering Module begins with a yellow speech bubble labeled 'question' pointing to a dynamic scene graph represented by interconnected white circles. This graph evolves through successive 'update' and 'search' operations, depicted by arrows labeled accordingly, leading to increasingly complex graphs. Each graph is associated with object position data stored in rectangular boxes labeled 'X_t, Y_t, and 'Bbox' (for bounding box), which are updated at each time step. These position boxes feed into yellow trapezoidal blocks labeled 'Mask-RCNN', indicating object detection and segmentation. The final output of this module is a blue speech bubble labeled 'answer', accompanied by a black silhouette of a person thinking, symbolizing human-like reasoning. The lower section, the Manipulation Module, starts with a sequence of four images showing a cluttered workspace with various objects and a robotic arm interacting with them. Each image feeds into a blue trapezoid labeled 'Visual CNN', which extracts visual features. Below each CNN block is a state sequence box containing three colored segments: 'a_{t-1}' (green), 't_{t}' (blue), and 'Q' (purple), representing action, location, and question context respectively. These state sequences are processed by green rectangular blocks labeled 'PLNR GRU', which form a recurrent network passing hidden states (h_t, h_{t+1}, etc.) from one time step to the next. The output of each PLNR GRU is a manipulation sequence box containing 'x', 'y', and 'o' values (coordinates and orientation), feeding into a green circular node labeled 'a_t', 'a_{t+1}', etc., representing actions. The final action node at time t+3 is labeled 'stop', indicating termination. To the far right of this module is a black robotic arm icon, visually representing the physical execution component. Connections between modules are shown via arrows: from the Visual CNN outputs to the PLNR GRU inputs, from PLNR GRU outputs to the manipulation sequence and action nodes, and from the action nodes back to the Mask-RCNN modules above, forming a feedback loop where manipulation updates the scene for subsequent question answering. The entire diagram emphasizes a tightly coupled system where visual perception, reasoning, and physical manipulation are iteratively coordinated over time.



Component (C)	0.518	0.714 +37.8%
Arrow (A)	0.150	0.650 +333.3%
Text (T)	0.568	0.591 +4.0%
Avg	0.412	0.652 +58.2%

Prompt: The figure illustrates the concept of unraveling a recurrent neural network (RNN) over time, showing how a compact, cyclic network structure is expanded into a sequence of interconnected time steps. On the left side, a compact RNN is depicted as a layered graph: the top layer consists of three black circular nodes labeled X(t), representing the input at time t; the middle layer contains four blue circular nodes enclosed in a light-blue rectangular box, symbolizing the hidden state or processing unit of the RNN; and the bottom layer has three black circular nodes labeled X(t-1), representing the previous time step's input. All nodes are fully connected with gray lines, indicating the recurrent connections within the network. The vertical label 'Recurrent Neural Network' runs along the left edge of this compact diagram. To the right of the compact RNN, an equality sign labeled 'Unraveled' indicates that the cyclic structure is unfolded into a linear sequence of time steps. This unfolded representation spans horizontally from left to right, aligned with a large light-blue arrow labeled 'Time' at the bottom, pointing rightward to indicate temporal progression. The unraveled structure consists of a chain of identical blue rectangular blocks labeled H□, H□, H□, ..., H□□, each representing the hidden state at a specific time step. Each block receives two inputs: one from below via a black circular node labeled with the corresponding input data (e.g., X(t-10) for H□, X(t-9) for H□, etc.), and one from the previous block in the chain (except for H□, which has no prior hidden state input). The output of each block is directed upward to a black circular node labeled with the predicted or computed output at that time step (e.g., X(t-9) from H□, X(t-8) from H□, etc.). Below each hidden state block, there is a descriptive caption indicating the data being processed at that time step: 'Data at time(t)' under H□, 'Data at time(t+1)' under H□, 'Data at time(t+2)' under H□, and so on, up to 'Data at time(t+n)' under H□□. The sequence continues with ellipsis (...) between H□ and H□□ to denote intermediate steps not explicitly drawn. The final block H□□ outputs to X(t), and an arrow extends from it to the right, suggesting continuation beyond the shown steps. The visual design uses consistent colors: black circles for inputs and outputs, blue rectangles for hidden states, and gray arrows for connections. The layout emphasizes the sequential, time-unfolded nature of RNNs, where each time step processes current input and the previous hidden state to produce a new hidden state and output.



Component (C)	0.827	0.923 +11.6%
Arrow (A)	0.536	0.786 +46.6%
Text (T)	0.500	1.000 +100.0%
Avg	0.621	0.903 +45.4%